03 MAR 2026

# AADK TASK 3

UI Item Interaction Design &
Stateful Favourite Feature

MindMatrix

## 1. Favourite Feature Design

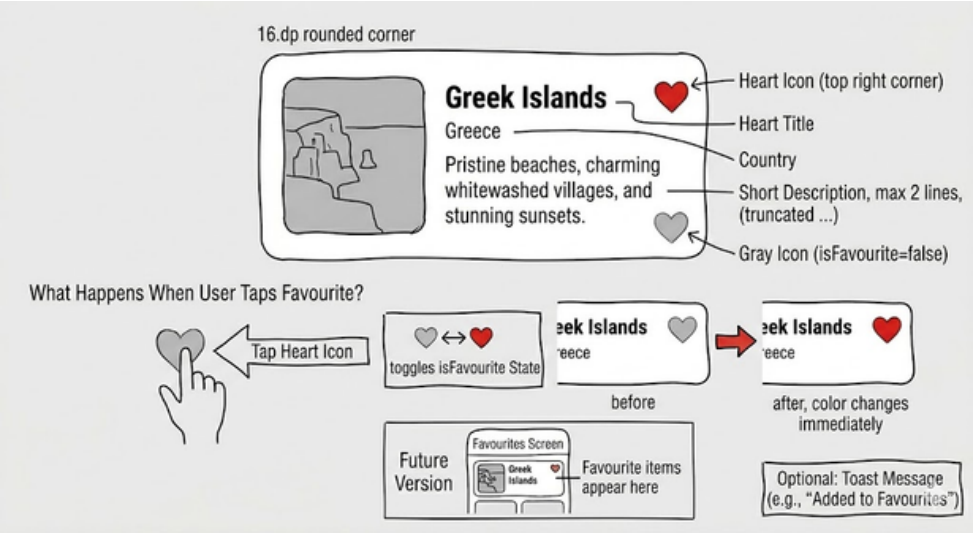
**Theme: Travel Destinations (Continuation from Task 1 & 2)**
In this task, I planned how my list items will respond when the user interacts with them, especially for Favourite toggle and item expansion. This task made me think more deeply about state management in Compose.

### UI Design Plan

Inside each list item Card:

- A Heart Icon will be placed at the top right corner.
- If isFavourite = false → icon color = Gray
- If isFavourite = true → icon color = Red
-

Layout idea:




### What Happens When User Taps Favourite?

- isFavourite state toggles (true ↔ false).
- Icon color changes immediately.
- Optional: Show small Toast message like
- "Added to Favourites" or "Removed from Favourites".
- In future version, favourite items can appear in a separate Favourites screen.

## 2. State Management Plan

This part is very important because LazyColumn reuses items.

### Option I Considered

var isFavourite by remember { mutableStateOf(false) }

Problem:

- State may reset when item recomposes.
- Scrolling up/down might lose favourite state.

### Final Approach I Will Use

I will:

- Store the entire list in a ViewModel
- Each item already has isFavourite property.
- When user clicks favourite:
  - Update the item inside the list in ViewModel.
  - UI automatically recomposes.

Example idea:

```
fun toggleFavourite(id: Int) {
    destinationList = destinationList.map {
        if (it.id == id) it.copy(isFavourite = !it.isFavourite)
        else it
    }
}
```

Why this is better:

- State survives recomposition.
- State survives scrolling.
- Can survive orientation change (if ViewModel is used).

## 3. Item Expansion Plan

### Interaction Design

When user taps the item (not heart icon):

- The Card expands.
- Full description becomes visible.
- Image size may increase slightly.

Collapsed View:

- Image
- Title
- Country
- 2 lines description

Expanded View:

- Image (slightly bigger)
- Full description
- Extra travel info (optional future feature)

### Interaction Design

I will use:

- var expanded by rememberSaveable { mutableStateOf(false) }
- animateContentSize() for smooth animation
- Conditional visibility:

```
if (expanded) {
   Text(fullDescription)
}
```

Why rememberSaveable?

- So expansion state does not reset during orientation change.

## 4. Error / Edge Case Handling

### Edge Case 1: User Quickly Toggles Favourite Many Times

Possible problem:

- UI lag or inconsistent state.

Solution:

- Since state is updated in ViewModel, only one source of truth exists.
- Disable double tapping for very short duration (optional improvement).

### Edge Case 2: LazyColumn Reuses Items

Problem:

- Item state may reset if managed locally.

Solution:

- State stored in ViewModel instead of inside composable.
- Use unique key in LazyColumn:

```
items(destinationList, key = { it.id })
```

This prevents incorrect state mapping.

### Edge Case 3: Orientation Change

Problem:

- Expanded item collapses after rotation.

Solution:

- Use rememberSaveable
- OR
- Store expanded item ID in ViewModel.

**Test Cases I Will Perform**

Scroll test:

- Mark item as favourite.
- Scroll down and up.
- Check if heart icon remains red.

Rotation test:

- Expand an item.
- Rotate screen.
- Check if it remains expanded.

Rapid click test:

- Click favourite icon 10 times quickly.
- Ensure state remains correct.

## 5. Reflection

Adding interactivity increases complexity because now we must manage state properly, not just design UI. Simple lists are easy, but when we add favourite toggling and expansion, we must think about recomposition, scrolling behavior, and configuration changes. There is a trade-off between simple UI and rich features. More features improve user experience but increase development effort. To ensure smooth experience, I will use ViewModel, proper state handling, and lazy components carefully. Planning before coding helps avoid many bugs related to state loss and incorrect UI updates.